

## Hafta 11 – Whitebox Kriptografi

|                   |            |
|-------------------|------------|
| Tarih             | 27.11.2026 |
| Öğrenme çıktıları | ÖÇ.2, 3    |
| Süre              | 3 saat     |



### Bu haftanın çalışan demosu

`code/week-11/01-oyuncak-tablo` – Oyuncak whitebox: kodlanmamış tablo anahtarı sizdir, kodlanmış tablo naif okumayı durdurur. · `code/week-11/02-gomulu-anahtar` – Gömülü (naif) anahtar ikilide fiziksel olarak bulunur → yazılımdaki anahtar korunmaz.

Çalıştırma: `sh demo.sh` (Linux/WSL) ya da CMake ile derleyip `bin/` altından. Tümüyle sentetik ve güvenlidir; öğrenci bilgisayarına zarar vermez.



### Demoyu kendiniz çalıştırın – adım adım (kopyala-yapıştır)

İlk derleme `code/README.md`'de anlatılır. `code` klasöründen:

```
# Windows (PowerShell)
.\build.ps1
cd week-11\01-oyuncak-tablo
.\bin\windows\oyuncak_wb.exe
cd ../02-gomulu-anahtar
.\bin\windows\gomulu.exe --tara
```

```
# WSL / Linux
./build.sh
cd week-11/01-oyuncak-tablo && ./bin/linux/oyuncak_wb
cd ../02-gomulu-anahtar && ./bin/linux/gomulu --tara
```

**Beklenen çıktı:** Naif tablo  $T[x]=S[x\oplus k]$  gizli anahtarı (  $0x3C$  ) **sızdırır**; kodlanmış tablo **sızdırmaz** (aynı şifreleme, farklı iç yapı). İkinci demo kendi ikilisine gömülü anahtarı **entropi taramasıyla** bulur → "diziye anahtar gömmek koruma değildir".

**Bu haftanın sonunda şunları yapabileceksiniz**

1. **Kara kutu, gri kutu ve beyaz kutu** saldırgan modellerini ayırmak; whitebox kriptografinin (WBC) çözmeye çalıştığı problemi — "anahtar ve kod aynı anda saldırganın elindeyken" — tanımlamak.
2. WBC saldırganın yeteneklerini (belleği okuma, akışı izleme, tek turu çalıştırma, hata enjeksiyonu) ve bir yazılım anahtarının neden **matematiksel olarak** gizlenemeyeceğini açıklamak.
3. **Tablo tabanlı** WBC gerçekleştirmesini (Chow AES) kavram düzeyinde çözümlmek: kısmi değerlendirme, iç ve dış kodlamalar, karıştırıcı bijeksiyonlar; ve bunun **maliyetini** (tablo boyutu, hız) tahmin etmek.
4. Yayınlanmış WBC tasarımlarının **saldırı geçmişini** (BGE, DCA, DFA) bir güvenlik kuralı olarak okumak: "saf yazılım WBC tek başına yeterli değildir".
5. WBC'yi **katmanlı savunmada doğru yere** koymak: gizleme, RASP, anahtar yenileme, cihaz/sürüm bağlama ve sunucu tarafı risk denetimiyle birlikte; ve donanım alternatiflerini (TEE, güvenli öge, HSM/SoftHSM) tanımak.

**Ders akışı (öğretim üyesi için zaman planı)**

| Zaman     | Bölüm | Ne yapılıyor                                                                                          |
|-----------|-------|-------------------------------------------------------------------------------------------------------|
| 0:00–0:25 | 1     | Kara/gri/beyaz kutu modelleri; WBC saldırgan yetenekleri; problem neden zor?                          |
| 0:25–0:50 | 1     | Naif çözümlerin çöküşü: gömülü anahtar, entropi saldırısı, kod taşıma (code lifting)                  |
| 0:50–0:55 | Ara   |                                                                                                       |
| 0:55–1:40 | 2     | Tablo tabanlı WBC (Chow AES): kısmi değerlendirme, iç/dış kodlama, bijeksiyonlar, tablo boyutu ve hız |
| 1:40–1:45 | Ara   |                                                                                                       |
| 1:45–2:20 | 3     | Saldırı geçmişi: BGE, DCA, DFA, WhibOx yarışmaları; "hepsi kırıldı" ve karşı önlemler                 |
| 2:20–2:45 | 4     | WBC'nin katmanlı savunmadaki yeri; donanım alternatifleri (TEE/SE, HSM/SoftHSM)                       |
| 2:45–3:00 | 5+    | Hatalı→saldırı→koruma örneği; proje S8; kendini sına                                                  |

**Bu haftanın kaynakları hakkında**

Bu hafta iki ana kaynağa dayanır: (1) **güvenli programlama teknik kılavuzu** — WBC saldırgan modelini ve bir üründe whitebox AES'in nasıl konumlandığını (anahtarların "güvence altına alınması ve/veya gizlenmesi" gereksinimi) anlatır; (2) alanın **temel akademik kaynakları** (Chow vd. 2002; kriptanaliz için BGE, DCA, DFA). WBC saldırgan yeteneklerini burada **savunmayı sınamak** için öğreniriz; çalışan saldırı kodu yazmayız. Bütün örnekler ve anahtarlar **sentetiktir**. Bu hafta 3. haftadaki "gizlilik kriptografinin işidir" ve 9. haftadaki "gizleme anahtarı korumaz" kurallarının doğal devamıdır.

## 0. Temel kavramlar (sıfırdan)

Bu bölüm **hiçbir ön bilgi varsaymaz**. Haftanın geri kalanında kullanacağımız terimleri sıfırdan tanımlıyoruz. Bir terimi bilmiyorsanız önce burayı okuyun; sonraki bölümler bunların üzerine kurulur.

### Şifreleme nedir?

- **Şifreleme (encryption)**: okunur veriyi (**açık metin**), bir **anahtar** kullanarak okunamaz hale (**şifreli metin**) getirmek.
- **Çözme (decryption)**: anahtarla geri açmak.

```
açık metin --(anahtar ile şifrele)--> şifreli metin
şifreli metin --(anahtar ile çöz)--> açık metin
```

### Anahtar nedir?

- **Anahtar (key)**: şifrelemeyi yöneten gizli sayı (bir bayt dizisi).
- Aynı algoritma + farklı anahtar = farklı sonuç.
- **Güvenlik anahtarın gizliliğine bağlıdır**, algoritmanın gizliliğine değil (Kerckhoffs ilkesi).

### Simetrik ve asimetrik

- **Simetrik**: şifreleme ve çözme **aynı** anahtar (ör. **AES**). Hızlı.
- **Asimetrik**: açık ve gizli anahtar çifti (ör. **RSA**). Yavaş ama anahtar dağıtımı kolay.

Bu hafta ağırlıklı **AES** (simetrik) üstünden gideceğiz.

### AES nedir? (yüksekten)

- **AES**: en yaygın simetrik şifreleme algoritması.
- 16 baytlık **blok**lar üzerinde çalışır.
- Veriyi **turlar** (rounds) halinde karıştırır; her turda: bayt değiştirme, satır/sütun karıştırma, anahtar ekleme.

Ayrıntı bu hafta sınavda sorulmayacak; **fikir** yeter.

### S-box nedir?

- **S-box (substitution box)**: bir baytı başka bir baytla değiştiren **sabit bir tablo**.
- AES'in "bayt değiştirme" adımı budur.
- Örnek:  $S\text{-box}[0x53] = 0xED$  gibi 256 girişli bir eşleme.

Bu tabloyu aklınızda tutun; whitebox'ın kalbi budur.

### Arama tablosu (lookup table)

- **Arama tablosu**: "girdiye karşılık çıktı" veren dizi.
- $T[x] = y$  için önceden hesaplanmış sonuç.
- Hesap yapmak yerine **tablodan okursunuz**; hızlıdır.

Whitebox, hesabı tablolara gömer.

## XOR hatırlatma

- **XOR** ( $\wedge$ ): bitler farklıysa 1, aynıysa 0.
- $a \wedge b \wedge b == a \rightarrow$  geri alınabilir.
- AES'te "anahtar ekleme" adımı bir XOR'dur:  $\text{durum} \wedge \text{anahtar}$ .

## Bit güvenlik düzeyi

- "**n-bit güvenlik**" = kırmak için kabaca  $2^n$  deneme gerekir.
- AES-128  $\rightarrow$  128 bit; bugün kırılmaz sayılır.
- Yüksek sayı = daha güçlü.

## Yan kanal (side channel) nedir?

- **Yan kanal:** algoritmanın matematiğini değil, çalışırken **sızdırdığı fiziksel/dolaylı bilgiyi** kullanan saldırı.
- Örnek: harcanan **zaman**, **güç** tüketimi, elektromanyetik yayım.

Kartlarda klasiktir; birazdan whitebox'ta yazılım karşılığını göreceğiz.

## DPA nedir?

- **DPA (Differential Power Analysis):** bir cihazın **güç tüketimi** ölçümlerine istatistik uygulayıp anahtarı çıkaran yan kanal saldırısı.
- Cihazın içine bakmaz; dışarıdan ölçer.

Bu fikri aklınızda tutun; whitebox'ın en güçlü saldırısı (DCA) bunun yazılım hâlidir.

## Bijeksiyon (birebir-örten eşleme)

- **Bijeksiyon:** her girdiyi tam bir çıktıya eşleyen, **tersi alınabilen** dönüşüm.
- Örnek:  $f(x) = x \wedge 0x5A$  bir bijeksiyondur (tersi yine kendisi).

Whitebox, tabloları **gizli bijeksiyonlarla** sarar.

## TEE ve güvenli öge (SE)

- **TEE (Trusted Execution Environment):** telefon işlemcisinin, işletim sisteminden **yalıtılmış** güvenli bölgesi.
- **Güvenli öge (SE):** anahtarları saklayan ayrı, kurcalamaya dirençli **donanım** çip.

Anahtar için en güçlü koruma bunlardır; whitebox bunların **olmadığı** durum içindir.

## HSM ve SoftHSM

- **HSM (Hardware Security Module):** sunucuda anahtarları saklayan/işleten özel donanım; anahtar **dışarı çıkmaz**.
- **SoftHSM:** HSM'in **yazılım benzetimi**; aynı arayüz ([PKCS#11](#)), ama donanım koruması yok (geliştirme/test için).

## Şimdi hazırız

Bildiğimiz terimler:

şifreleme/çözme · anahtar · simetrik/asimetrik · AES · S-box · arama tablosu · XOR · bit güvenlik · yan kanal · DPA · bijeksiyon · TEE/SE · HSM/SoftHSM

Şimdi asıl soruya: **anahtar saldırısının elindeyken ne yaparız?**

## 1. Üç saldırı modeli: kara, gri, beyaz kutu

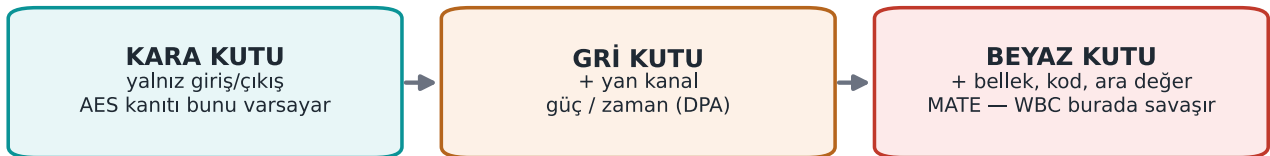
Kriptografiyi 3. ve 10. haftalarda **kara kutu** varsayımıyla ele aldık: saldırı yalnız girdiyi ve çıktığı görür, anahtarı ve algoritmanın iç durumunu göremez. AES, RSA, HMAC gibi standart algoritmaların güvenlik kanıtları bu varsayıma dayanır. Ama bu varsayım, kodu kullanıcıya teslim ettiğimizde çöker.

### ✎ Kısa tarihçe: whitebox kriptografi nereden geldi?

- **1883** — Kerckhoffs ilkesi: güvenlik **anahtarda** olmalı, sistemin gizliliğinde değil. Beyaz kutu düşüncesinin kökü budur.
- **2002** — Chow, Eisen, Johnson, van Oorschot ilk **whitebox AES** ve **whitebox DES**'i (DRM için) yayımlar. WBC'nin doğuşu.
- **2004** — Billet–Gilbert–Ech–Chatbi (**BGE saldırısı**) ilk WB-AES'i kırar → "hepsi kırılır" kuralının başlangıcı.
- **2016** — Bos vd. **DCA** (Differential Computation Analysis): donanımdaki DPA'yı yazılıma taşır, birçok WBC'yi **otomatik** kırar.
- **2017–2024** — **WhibOx** yarışmalarında yayımlanan tüm saf-yazılım WB-AES adayları kırıldı → bugünkü kural: WBC bir **geciktirme katmanıdır**; mümkünse donanım (TEE/SE/HSM) tercih edilir.

## Üç saldırı modeli: ne görüyor?

soldan sağa görünürlük artar



WBC en zor modelde anahtar çıkarmayı **GECİKTİRMEYE** çalışır; imkânsız kılmaz.

Görünürlük soldan sağa artar; WBC en zor (beyaz kutu) modelde anahtar çıkarmayı **geciktirmeye** çalışır.

| Model      | Saldırgan neyi görür/yapar?                                       | Tipik ortam                                       |
|------------|-------------------------------------------------------------------|---------------------------------------------------|
| Kara kutu  | Yalnız girdi ve çıktı                                             | Uzak sunucu; ağ saldırıları                       |
| Gri kutu   | Girdi/çıkış + <b>yan kanallar</b> (zaman, güç, elektromanyetik)   | Akıllı kart, gömülü cihaz (yan kanal saldırıları) |
| Beyaz kutu | <b>Her şey</b> : kod, bellek, ara değerler; ayrıca değiştirebilir | Yazılım, saldırganın kendi cihazında              |

Whitebox kriptografi (WBC), **beyaz kutu** ortamda çalışan bir şifreleme algoritmasının güvenlik sorunlarını inceler. Kaynağın diliyle: WBC saldırgan modeli Chow ve arkadaşları tarafından 2002'de tanımlanmıştır ve saldırganın şu yetenekleri vardır (bunları savunmanın **karşı koyması gerekenler** olarak okuyun):

- **Çalışmayı gözlemleme**: o an işlenen komutlara erişim, algoritma akışını izleme, kullanılan belleği görme.
- **Ortamı denetleme / çalışma anında değiştirme**: program belleğini kurcalama, algoritmanın **yalnız bir parçasını** (ör. şifrenin bir turunu) değiştirme, if koşullarını değiştirme, döngü sayaçlarını değiştirme, **hata enjekte etme**.

### ” Kaynağın gereksinimi

Teknik kılavuz bu bölümü bir uyum maddesiyle açar: *"Kriptografik anahtarlar, gizlilik ve bütünlüğü korumak için güvence altına alınmalı ve/veya gizlenmelidir."* Ama hemen ekler: bu saldırgan yeteneklerine karşı uygulamayı ve varlıkları koruyan şey **yalnız WBC değil, kod sağlamlaştırma (9. hafta) ve RASP (6. hafta) yöntemleridir**. Yani WBC daha ilk cümlede **tek başına bir çözüm olarak sunulmaz**.

## 2. Problem neden bu kadar zor?

Beyaz kutuda temel gerilim şudur: program şifrelemeyi yapabilmek için anahtara **erişebilmek** zorundadır; ama saldırgan da programın gördüğü her şeyi görür. Yani anahtar, bir noktada, saldırganın da görebildiği bir yerdedir. WBC'nin fikri, anahtarı **hiçbir zaman açıkça var olmayacak** biçimde algoritmanın içine "pişirmektir": anahtar, önceden hesaplanmış **arama tablolarının** içine gömülür, böylece bellekte "işte anahtar bu" diyebileceğiniz bir bayt dizisi bulunmaz.

Bu, gizliliği kriptografiden gizlemeye taşımaz; **matematiksel bir dönüşümle** anahtarı tablolara dağıtmaya çalışır. Amaç yine bir **maliyet kuralıdır** (9. haftadaki gibi): anahtarı tablolardan geri çıkarmayı, onu düz saklamaktan çok daha pahalı hale getirmek.

### Naif çözümlerin neden çöktüğü (kurallar)

WBC'ye neden ihtiyaç olduğunu görmek için, işe yaramayan "kolay" çözümleri ve onları kıran saldırıları bilmek gerekir. Her biri bir güvenli programlama kuralına dönüşür:

- **Kural — Anahtarı sabit diziye gömme**. En naif çözüm anahtarı bir `static const uint8_t key[16]` dizisine koymaktır. Bu, `strings` ve **entropi taraması** (Shamir–van Someren'in gözlemi: kriptografik anahtarlar yüksek entropili bloklardır ve ikili dosyada öyle görünürler) ile saniyeler içinde bulunur; bir hata ayıklayıcıyla doğrulanır. **Sonuç**: düz gömülü anahtar bir anahtar koruması değildir.
- **Kural — Anahtarı yalnız XOR/whitening ile "karıştırma"**. Anahtarı bir maskeyle XOR'layıp saklamak da işe yaramaz: maske de ikili dosyadadır, program onu çözebiliyorsa saldırgan da çözer.
- **Kod taşıma (code lifting) tehdidi**. WBC anahtarı tablolara gömse bile, saldırgan **anahtarı çıkarmadan** şifreleme/çözme yapan kod parçasını (tablolar + yorumlayıcı) olduğu gibi kopyalayıp kendi programında kullanabilir. Anahtarı

hiç bilmeden onun işlevini çalar. **Sonuç:** WBC'nin tek başına "anahtarı gizledim" demesi yetmez; kod taşımaya karşı **cihaz/sürüm bağlama** (bölüm 4) gerekir.



#### Bu haftanın ana kuralı, baştan

Yayımlanmış hiçbir saf yazılım whitebox tasarımı bugüne kadar kırılmadan kalmamıştır. O yüzden WBC'yi "**anahtarı güvene alan sihir**" olarak değil, "**anahtar çıkarmayı geciktiren ve diğer katmanlarla birlikte anlam kazanan bir katman**" olarak öğreneceğiz. Bu, 9. haftanın gizleme kurallarının kriptografiye uygulanmış halidir.

### 3. Tablo tabanlı WBC nasıl çalışır? (Chow AES, kavram)

WBC'nin klasik fikri, Chow ve arkadaşlarının 2002'deki AES gerçekleştirmesidir. Ayrıntılı matematiği (sonlu cisim işlemleri, matris kurulumları) bu dersin sınavında sorulmayacak; ama **fikri** ve özellikle **maliyetini** ve **sınırını** anlamamız gerekir. Adım adım ve kavramsal gidiyoruz.

#### Adım 1 — Kısmi değerlendirme: anahtarı tabloya "pişir"

AES bir turda önce durum baytını tur anahtarıyla XOR'lar, sonra S-box'tan geçirir. Bu iki işlem, sabit bir anahtar için **tek bir arama tablosuna** birleştirilebilir:

|         |                                                   |                                            |
|---------|---------------------------------------------------|--------------------------------------------|
| Normal: | $\text{çıkıkt1} = \text{S-box}[x \text{ XOR } k]$ | (k tur anahtarının baytı)                  |
| WBC:    | $T[x] = \text{S-box}[x \text{ XOR } k]$           | → 256 girişli bir tablo, k tablonun İÇİNDE |

Artık koddaki  $k$  diye bir değişken yoktur;  $k$  tablonun içeriğine gömülmüştür. Ama bu **tek başına güvensizdir**: saldırgan tabloyu S-box ile karşılaştırıp  $k$ 'yı geri çıkarabilir (tablo, S-box'ın  $x \text{ XOR } k$  ile ötelenmiş halidir). O yüzden tablolar **kodlanır**.

#### Sayısal mikro-örnek: anahtar tablodan nasıl sızıyor? (uçtan uca)

Yukarıdaki cümle ("saldırgan  $k$ 'yı geri çıkarabilir") havada kalmasın; **gerçek sayılarla, tek tek** yapalım. Elimizde yalnız şunlar var: herkesin bildiği **AES S-box'**ı ve saldırganın ikili dosyadan okuduğu **tablo**.

**Kurulum.** Gizli anahtar baytı  $k = 0x3C$  olsun (saldırgan bunu **bilmiyor**). Geliştirici Adım 1'i uygulayıp şu tabloyu üretmiş:

$T[x] = \text{S-box}[x \text{ XOR } k]$ .

**Adım 1 — Tablodan tek bir değer oku.** Saldırgan  $x = 0x00$  girişine bakar:

$T[0x00] = \text{S-box}[0x00 \text{ XOR } 0x3C] = \text{S-box}[0x3C] = 0xEB$

Yani ikili dosyada okuduğu değer:  $T[0x00] = 0xEB$ .

**Adım 2 — S-box'ı tersine çevir.** S-box herkese açık ve **birebirdir**; tersi de bilinir:

$\text{S-box}^{-1}[0xEB] = 0x3C$  (çünkü  $\text{S-box}[0x3C] = 0xEB$ )

**Adım 3 — Anahtarı çöz.** Tanım gereği  $\text{S-box}^{-1}[T[x]] = x \text{ XOR } k$  olduğundan:

```
x XOR k = 0x3C
0x00 XOR k = 0x3C
k = 0x3C XOR 0x00 = 0x3C      ← ANAHTAR BULUNDU
```

**Sonuç.** Saldırgan **tek bir tablo girdisiyle**, 256 deneme bile yapmadan anahtarı buldu. "Anahtar kodda görünmüyor" demek, "anahtar korunuyor" demek **değildir**.



#### Çıkarılan kural

Kısmi değerlendirme **tek başına** anahtarı gizlemez; çünkü tablo, bilinen S-box'ın  $k$  kadar **ötelenmiş** hâlidir ve bu öteleme geri alınabilir. Tabloyu güvenli kılan şey Adım 3'teki **iç kodlamalardır**.

**Peki kodlama ne değiştirir?** Tabloyu gizli bir bijeksiyon  $G$  ile sararsak:  $T'[x] = G(S\text{-box}[x \oplus k])$ . Saldırgan artık  $S\text{-box}^{-1}[T'[x]]$  hesaplayamaz, çünkü önce  $G$ 'yi kaldırması gerekir ama  $G$ 'yi bilmiyor. Adım 2 kırılır, dolayısıyla Adım 3 de çalışmaz.



#### Bunu demoda kendi gözünüzle görün

`code/week-11/01-oyuncak-tablo` demosu tam bu iki tabloyu üretir: **naif** tabloda yukarıdaki üç adım anahtarı ( $0x3C$ ) bulur, **kodlanmış** tabloda aynı adımlar başarısız olur. Çalıştırma komutları bu sayfanın başındaki "Demoyu kendiniz çalıştırın" kutusunda.

## Adım 2 — Tabloları birleştir ve genişlet (T-box + MixColumns)

Tek bir bayt eşlemesi yerine, S-box ile AES'in doğrusal karıştırma adımı (MixColumns) birleştirilir; 8 bitlik girdiden 32 bitlik çıktı üreten daha büyük tablolar (T-box'lar) elde edilir. Baytlar arası XOR işlemleri de küçük **XOR tabloları** (4 bitlik) haline getirilir. Böylece bütün tur, bir **arama tabloları ağına** dönüşür; hiçbir yerde açık bir aritmetik anahtar işlemi görünmez.

## Adım 3 — İç kodlamalar (internal encodings)

Her tablonun çıkışına gizli bir bijeksiyon (birebir-örten eşleme) uygulanır ve bir sonraki tablonun girişinde onun tersi uygulanır. Ardışık tabloların kodlamaları birbirini götürür; sonuç doğrudur ama **ara değerler karışıktır**. Bu, bir tabloyu tek başına inceleyen saldırıyı durdurmayı amaçlar.

## Adım 4 — Karıştırıcı bijeksiyonlar (mixing bijections)

İç kodlamalara ek olarak,  $GF(2)$  üzerinde doğrusal karıştırıcı matrisler (ör.  $8 \times 8$  ve  $32 \times 32$ ) eklenir; bunlar bilgiyi tablolar arasında **yayar**, böylece bir tablodan sızan bilgi tek başına anlamlı olmaz. Chow'un tablo tipleri (literatürde I–IV) bu kodlanmış ve karıştırılmış tabloların türleridir.

## Adım 5 — Dış kodlamalar (external encodings): F ve G

En dış katmanda, tüm şifre iki gizli bijeksiyonla sarılır. Kaynak kılavuzun anlattığı gibi, whitebox AES işlemi **F ve G** adlı iki rastgele tekil-olmayan matrisle kapsülendir (128 rank, "rastgele bijeksiyon"). Yani ağ artık saf AES değil,  $G \circ AES \circ F^{-1}$  hesaplar:

```
Girdi --F-1--> [ kodlanmış AES tabloları ağı ] --G--> Çıktı
```

**Bu adım hem en güçlü korumadır hem de en büyük sınırdır:**

- **Güç:** Dış kodlama, tek başına tablolara bakarak yapılan pek çok cebirsel/istatistiksel saldırıyı (BGE, DCA türevleri) çok zorlaştırır; çünkü saldırganın gördüğü giriş/çıkış standart AES'in giriş/çıkışı değildir.
- **Sınır:**  $G \circ AES \circ F^{-1}$  **standart AES değildir.** Karşı tarafın (sunucu ya da başka bir uç)  $F$  ve  $G$ 'yi bilip geri almadan sonuç anlamsızdır. Bu yüzden dış kodlamalı WBC, ancak **iki ucu da sizin kontrolünüzde olan** bir protokolde kullanılabilir; EMV gibi standart AES bekleyen bir protokolde kullanılamayabilir.

**Maliyet: WBC bedava değildir**

Kavramı bir kural olarak bitirelim: WBC'nin maliyeti yüksektir. Chow AES'in tipik büyüklük mertebesi (literatürden):

| Ölçüt                    | Yaklaşık değer (meretebe)                                                   |
|--------------------------|-----------------------------------------------------------------------------|
| Tablo boyutu             | Yüzlerce KB (ör. ~770 KB) — karşılaştırma: standart AES anahtarı 16–32 bayt |
| Tur başına tablo okuması | Binlerce arama (ör. ~3000)                                                  |
| Hız                      | Standart AES'e göre onlarca kat yavaş (ör. ~50×)                            |

**Sınav için ölçü hissi**

Bir tabloların boyutunu tahmin edebilmelisiniz: ör. 288 tablo  $\times$  1 KB  $\approx$  294.912 bayt. Amaç kesin sayı değil, **meritebeyi** ve bunun neden yalnız seçili, küçük ve kritik işlemler için kullanıldığını görmektir. Bu, 9. haftadaki sanallaştırma kuralının (K-10) maliyet mantığının aynısıdır.

## 4. Saldırı geçmişi: neden "hepsi kırıldı" bir kuraldır

WBC'yi doğru konumlandırmak için en dürüst yol, tarihine bakmaktır. Aşağıdaki kronolojiyi **saldırı öğretmek için değil**, "bu korumaya ne kadar güvenebilirim?" sorusunu yanıtlamak için veriyoruz. Her satır, savunmacı için bir karşı önlem çıkarımıdır.

| Yıl       | Kilometre taşı                                                                                                                                        | Savunmacı için çıkarım                                       |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| 2002      | Chow vd. WB-AES ve WB-DES tasarımları                                                                                                                 | WBC fikrinin doğuşu; tablo tabanlı yaklaşım                  |
| 2004      | <b>BGE saldırısı</b> (Billet–Gilbert–Ech–Chatbi): Chow AES pratik sürede kırıldı ( $\sim 2^{30}$ )                                                    | İç kodlamalar tek başına yetmez                              |
| 2007      | Goubin/Wyseur vd.: WB-DES ve harici kodlamalı türevler kırıldı                                                                                        | DES tabanlı WBC bırakılır                                    |
| 2010'lar  | Karroumi'nin "çift AES" güçlendirmesi de kırıldı ( $\sim 2^{22}$ )                                                                                    | "Daha çok tablo" güç demek değildir                          |
| 2015      | <b>DFA</b> ("Unboxing the White-Box", Black Hat EU): hata enjeksiyonu, tasarımın iç matematiğini bilmeden anahtarı çıkarır                            | Genel (gri kutu) saldırılar WBC'ye de uygulanır              |
| 2016      | <b>DCA</b> (Differential Computation Analysis, CHES): yürütme izlerine güç analizi (DPA) benzeri istatistik uygulanır; iç kodlamaları görmezden gelir | Klasik yan kanal savunmaları WBC için de gerekli             |
| 2017–2024 | <b>WhibOx</b> yarışmaları: akademi ve endüstrinin gönderdiği tasarımların <b>tamamı</b> kırıldı                                                       | Bugün için: kırılmadan kalan yayımlanmış saf-yazılım WBC yok |

**DCA ve DFA neden önemli?** Çünkü bunlar tasarımın gizli matematiğini çözmeye çalışmaz. DCA, çalışırken üretilen ara değerlerin izini toplar ve tıpkı bir donanım güç analizinde (DPA) olduğu gibi istatistiksel korelasyonla anahtarı çıkarır; iç kodlamalar bu istatistiği çoğu zaman bozmaz. DFA ise çalışma anında bir değere **hata enjekte edip** bozulan çıktıdan anahtarı geri hesaplar. İkisi de "kutuyu açmadan" çalışır — bu yüzden yeni tasarımları da vurur.

## Karşı önlemler (savunma kuralları)

Araştırma durmadı; DCA/DFA'ye karşı önlemler geliştirildi. Ama hiçbirisi "artık güvenli" demez; hepsi **maliyeti artırır**:

- **Statik rastgelelik ve doğrusal olmayan maskeleyme** (ör. Biryukov–Udovenko 2018): DCA'nın dayandığı doğrusal korelasyonu bozmaya çalışır.
- **Dış kodlama (F, G)**: DCA/DFA/BGE'yi zorlaştırır — ama gördüğümüz sınırla: standart AES uyumluluğunu kırar.
- **Katmanlı savunma**: WBC'nin çevresine 9. haftanın gizleme kuralları, 6. haftanın RASP'ı (anti-debug, kurcalama ve bütünlük denetimi) konur; amaç, saldırganın izleri toplamasını (DCA) ve hata enjekte etmesini (DFA) zorlaştırmaktır.
- **Anahtar yenileme ve kısa ömür**: Anahtar sık yenilenirse, bir anahtarı çıkarmanın değeri düşer (10. hafta, kripto-periyot).
- **Cihaz/sürüm bağlama**: Kod taşımaya (code lifting) karşı; tablolar yalnız belirli bir cihazda/sürümde anlamlı olur.
- **Sunucu tarafı risk denetimi**: Uçtaki koruma ne olursa olsun, işlemin sunucuda ayrıca değerlendirilmesi (hız sınırı, anomali, ATC/sayaç denetimi) son ve en güvenilir savunmadır.
- **Donanım köküne taşıma**: Mümkünse anahtarı hiç yazılıma koymamak — TEE, güvenli öge (SE), HSM (bölüm 4).

**Ana kural (tekrar): WBC bir katmandır, çözüm değil**

Kaynak kılavuzun ilk cümlesini hatırlayın: WBC saldırgan yeteneklerine karşı uygulamayı koruyan şey **kod sağlamlaştırma ve RASP ile birlikte** WBC'dir. WBC'yi tek başına "anahtar güvene aldım" diye sunmak, bu dersteki en ciddi değerlendirme hatasıdır. Doğru ifade: "anahtar çıkarmayı şu kadar geciktiriyorum; asıl güvencem anahtar yenileme, cihaz bağlama ve sunucu risk denetiminde."

## 5. WBC'nin katmanlı savunmadaki yeri

Bir güvenli programlama kuralı olarak WBC'yi nereye koyacağımızı netleştirelim. Aşağıdaki tablo, bir anahtar korumak için seçeneklerin **güç/maliyet** sırasını verir:

| Seçenek                        | Anahtar nerede?                             | Güç               | Ne zaman?                                                 |
|--------------------------------|---------------------------------------------|-------------------|-----------------------------------------------------------|
| Sabit dizide düz anahtar       | Yazılımda, açık                             | Yok               | Asla (naif)                                               |
| Gizlenmiş/parçalanmış anahtar  | Yazılımda, gizlenmiş                        | Çok düşük         | Yalnız düşük değerli, kısa ömürlü sırlar                  |
| <b>WBC + katmanlı savunma</b>  | Yazılımda, tablolara gömülü                 | Orta (geciktirir) | Donanım kökü yoksa; anahtar yenileme + sunucu denetimiyle |
| <b>TEE / Güvenli Öğe (SE)</b>  | Donanımla yalıtılmış                        | Yüksek            | Cihaz destekliyorsa tercih edilir                         |
| <b>HSM / SoftHSM (PKCS#11)</b> | Donanım (ya da yazılım benzetimi) modülünde | Yüksek            | Sunucu tarafı; anahtar hiç dışarı çıkmaz                  |

**Karar kuralı**

**Anahtarı yazılıma koymak zorunda değilseniz, koymayın.** Donanım kökü (TEE/SE/HSM) varsa tercih edin. WBC, donanım kökünün **olmadığı** ya da her cihaza güvenilemediği durumlarda, anahtar çıkarmayı geciktiren bir **ödünleşim** çözümüdür — ve her zaman anahtar yenileme, cihaz/sürüm bağlama ve sunucu risk denetimiyle birlikte.

## SoftHSM ve PKCS#11 (sunucu tarafı köprü, 10. haftadan)

Onuncu haftada anahtar saklama için **PKCS#11** ve SoftHSM'den söz etmiştik. Sunucu tarafında anahtar, standart bir **PKCS#11** arayüzü ardında tutulur; uygulama anahtarın değerini hiç görmez, yalnız "şunu imzala/şifrele" der. SoftHSM, gerçek bir HSM'in yazılım benzetimidir; **aynı arayüzü** sunar, bu yüzden geliştirme ve test için kullanılabilir ama gerçek donanım koruması sağlamaz. WBC'nin uçtaki (istemci) problemine karşılık, bu sunucudaki anahtar problemine yanıtır.

## 6. Hatalı → saldırı → koruma (özet senaryo)

Bu haftanın kurallarını tek bir hikâyede toplayalım (sentetik, savunma amaçlı):

1. **Hatalı tasarım:** İstemci uygulaması bir veri şifreleme anahtarını `static const uint8_t k[16]` olarak gömer.
2. **Saldırı (kavramsal):** `strings` ve entropi taraması yüksek entropili 16 baytlık bloğu bulur; bir hata ayıklayıcı, şifreleme çağrısında bu bloğun anahtar olarak kullanıldığını doğrular. Anahtar dakikalar içinde çıkar.
3. **Koruma (katmanlı):**
4. Anahtarı mümkünse **TEE/SE**'ye taşı; değilse **WBC tablolarına** göm (düz bayt dizisi kalmamasın).
5. WBC'yi 9. haftanın **gizleme** kurallarıyla ve 6. haftanın **RASP**'ıyla (anti-debug, kurcalama tespiti) sar; böylece DCA izleri toplamak ve DFA hata enjekte etmek zorlaşır.
6. **Cihaz/sürüm bağla:** tablolar başka cihaza kopyalanınca (code lifting) işe yaramasın.
7. **Anahtarı sık yenile** (kısa krypto-periyot) ve **sunucuda risk denetimi** uygula.
8. **Dürüst kalan risk:** Yeterince kararlı bir saldırgan tek bir cihazda anahtarı yine de çıkarabilir; ama artık bir kopyayı kırmak bütün sistemi açmaz ve çıkarılan anahtarın ömrü kısadır.

## 7. Dönem projesi: bu hafta (S8 — anahtar koruma gerekçesi)

Projenizin S8 (kripto ve anahtarlar) bölümüne **en az bir hassas anahtar** için bir koruma kararı ve **gerekçesi** ekleyin:

1. Anahtarın nerede durduğu, nasıl korunduğu (düz saklanmıyorsa: TEE/SE, WBC, gizleme — hangisi ve neden).
2. Koruma yeterli değilse (ör. saf yazılımdaysanız) bunu **açıkça** yazın ve telafi eden katmanları belirtin: anahtar yenileme periyodu, cihaz/sürüm bağlama, sunucu risk denetimi.
3. Kullanmıyorsanız (ör. WBC uygulamadıysanız) **neden** kullanmadığının gerekçesi (maliyet, donanım kökü var, varlık değeri düşük). Gerekçeli bir "kullanmadık" tam bir yanittir.

### ⚠ Depoda gerçek anahtar bulunmasın

Bütün anahtar dosyaları, tablolar ve örnekler **sentetik** olmalıdır. Bir sertifikasyon değerlendirmesinde depoda gerçek bir anahtar bulunması en ağır bulgulardan biridir; projede de öyle sayılır.

## 8. Kendini sinama

### ? 1. Kara, gri ve beyaz kutu saldırgan modellerini bir cümleyle ayırın. Standart AES'in güvenlik kanıtı hangisini varsayar? ✓

**Kara kutu:** saldırgan yalnız giriş/çıkışı görür. **Gri kutu:** ayrıca yan kanalı (güç/zaman) gözler. **Beyaz kutu:** belleğe, koda ve ara değerlere tam erişimi vardır. Standart AES'in güvenlik kanıtı **kara kutu** modelini varsayar.

### ? 2. WBC saldırganın 'yalnız bir turu çalıştırma' ve 'hata enjekte etme' yeteneği hangi saldırılara (DCA/DFA) zemin hazırlar? ✓

Ara değerleri izleyip istatistiksel çözmek **DCA**'ya (Differential Computation Analysis), kasıtlı hata verip çıktığı karşılaştırmak **DFA**'ya (Differential Fault Analysis) zemin hazırlar.

### ? 3. Anahtarı sabit diziye gömmek neden koruma değildir? Entropi taraması burada ne işe yarar? ✓

Anahtar, ikili içinde **yüksek entropili** bir bayt bloğu olarak durur; kodun içine koymak yalnızca konumunu değiştirir, gizlemez. Entropi taraması ikilideki yüksek-entropili (anahtar benzeri) bölgeleri bulur; demo --tara bunu gösterir.

### ? 4. Kod taşıma (code lifting) nedir? WBC, anahtarı hiç bilmeyen bir saldırgana karşı neden savunmasız olabilir? Hangi önlem bunu kapatır? ✓

**Code lifting:** saldırgan whitebox tabloyu/rutini olduğu gibi kopyalayıp başka bir yerde kullanır — anahtarı hiç çıkarmadan şifreleme/çözme yapar. Bunu **dış kodlama (external encoding)** kapatır: fonksiyonun giriş/çıkışı uygulamaya bağlanır, taşınan rutin başka bağlamda işe yaramaz (ek olarak cihaz bağlama).

### ? 5. Kısmi değerlendirme $T[x] = S\text{-box}[x \oplus k]$ tek başına neden güvensizdir? ✓

Tablo doğrudan  $k$ 'ye bağlıdır; saldırgan tabloyu bilinen S-box ile karşılaştırıp  $x \oplus k$  eşlemesinden  $k$ 'yi geri hesaplar. Güvenli olması için tablo giriş/çıkışları gizli iç kodlamalarla maskelenmelidir.

### ? 6. İç kodlamalar ile dış kodlamalar (F, G) arasındaki fark nedir? Dış kodlamanın en büyük sınırı nedir? ✓

**İç kodlamalar** tablolar arası ara değerleri gizli bijeksiyonlarla maskeler (bileşen içi). **Dış kodlamalar (F, G)** fonksiyonun giriş ve çıkışını dönüştürür ve uygulamaya bağlar (code lifting'i zorlaştırır). En büyük sınır: giriş/çıkış değiştiği için **çağırın tarafın da eşleşmesi** gerekir → çıktı artık standart AES değildir, her yerde kullanılamaz.

### ? 7. Yaklaşık tablo boyutunu ve hız maliyetini mertebe olarak söyleyin. Bu neden WBC'yi yalnız seçili işlemlerle sınırlar? ✓

Tablolaştırılmış WB-AES tabloları **megabayt** mertebesinde ve kara kutu AES'e göre **onlarca-yüzlerce kat** yavaştır. Bu boyut/hız cezası yüzünden WBC yalnız seçili, kritik işlemlere uygulanır; tüm krypto akışına değil.

### ? 8. 'Yayımlanmış bütün saf-yazılım WBC tasarımları kırıldı' ifadesi, WBC'yi projede kullanma kararınızı nasıl etkiler? ✓

WBC'yi **tek başına** anahtar koruması olarak görmeyiz; onu anahtar çıkarmayı **geciktiren bir katman** olarak, anahtar yenileme + cihaz bağlama + sunucu tarafı denetimle birlikte kullanırız. Mümkünse donanım destekli koruma (TEE/SE/HSM) tercih edilir.

### ? 9. DCA klasik hangi donanım saldırısına benzer? İç kodlamalar neden onu her zaman durdurmaz? ✓

**DPa**'ya (Differential Power Analysis) benzer; DCA güç izinin yerine **yazılım izini** (bellek erişimleri/ara değerler) koyar. İç kodlamalar doğrusal ya da zayıfsa istatistiksel korelasyon kodlamayı aşabilir; bu yüzden DCA'yı her zaman durdurmaz.

10. Bir anahtarı korumak için seçenekleri (düz → gizli → WBC → TEE/SE → HSM) güç/maliyet ekseninde sıralayın.

Zayıf/ucuzdan güçlü/pahalıya: **düz gömülü anahtar < gizlenmiş/kodlanmış tablo (saf-yazılım WBC) < WBC + bağlama/yenileme < TEE/güvenli öge (SE) < HSM/donanım**. Güç ve maliyet aynı yönde artar; seçim varlık değerine göre yapılır.

11. WBC kullanıyorsanız, onu tek başına bırakmamak için eklemeniz gereken en az üç katmanı sayın.

**Anahtar yenileme (rotation), cihaz/uygulama bağlama (binding) ve sunucu tarafı kullanım denetimi/iptal**. Buna RASP/bütünlük denetimi de eklenir.

12. Projenizdeki bir anahtar için S8 koruma gerekçesini üç cümleyle yazın.

Örnek: 'Bu DEK, beklemedeki kullanıcı verisini AES-256-GCM ile korur. Anahtar **SoftHSM/PKCS#11** ile üretilip saklanır ve düz biçimde bellekte tutulmaz. Saf-yazılım whitebox tasarımları kırıldığı ve varlık değeri yüksek olduğu için donanım destekli koruma seçilmiştir.'

## 9. Kaynaklar ve ileri okuma

- **Güvenli programlama teknik kılavuzu** (ders kaynağı) — WBC saldırgan modeli ve whitebox AES'in bir üründe konumu; "anahtarlar güvence altına alınmalı ve/veya gizlenmelidir" gereksinimi.
- S. Chow, P. Eisen, H. Johnson, P. van Oorschot — "White-Box Cryptography and an AES Implementation" (SAC 2002) ve WB-DES (DRM 2002) — tablo tabanlı WBC'nin temeli.
- B. Wyseur — "White-Box Cryptography: Hiding Keys in Software" (kısaca giriş) — öğrenciye uygun genel bakış.
- Billet, Gilbert, Ech-Chatbi — BGE saldırısı (SAC 2004).
- Bos, Hubain, Michiels, Teuwen — "Differential Computation Analysis" (CHES 2016).
- Sanfeli, Mune, de Haas — "Unboxing the White-Box" (Black Hat EU 2015) — DFA/DPA.
- **WhibOx** yarışması sonuçları (2017–2024) — yayımlanmış tasarımların durumu.

### i Bir sonraki hafta

**12. hafta — Sertifikasyon ve sızma testi planlaması.** Bu hafta bir korumanın "ne kadar dayandığını" ölçmenin önemini gördük; 12. haftada bunu bağımsız bir değerlendirmenin süreç ve raporlamasına bağlayacağız (saldırı potansiyeli puanlaması, 13. hafta).